



Chương 4: Hệ thống Bus

Computer Engineering Faculty

Embedded Systems

Nội Dung



1

CPU Bus và I/O trên bus

2

Cấu trúc và phân loại hệ thống bộ nhớ

3

Các thiết bị nhập xuất I/O

2

Serial và Parallel I/O

3

PWM và GPIO

2

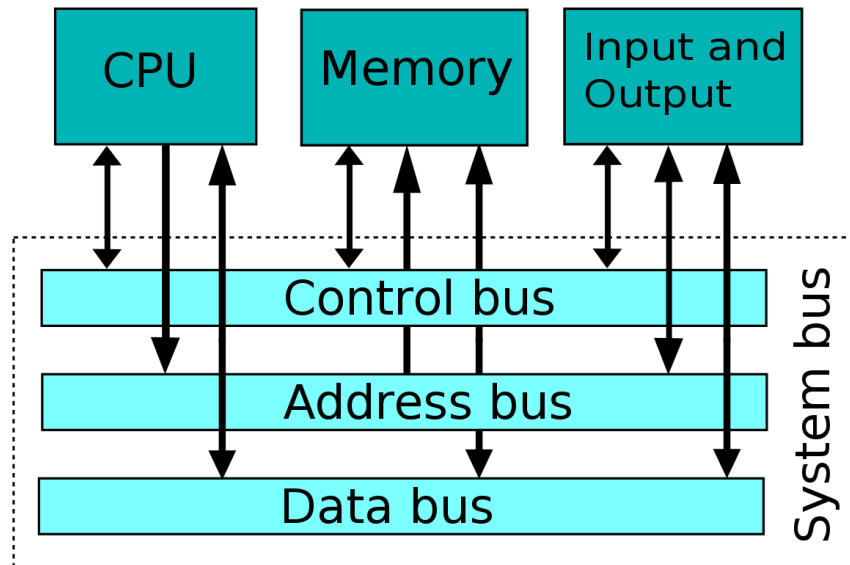
Các kỹ thuật thiết kế và debug

CPU Bus và I/O trên bus



□ Bus địa chỉ

- Các đường dẫn tín hiệu logic một chiều truyền địa chỉ tham chiếu bộ nhớ.
- Địa chỉ truy nhập trực tiếp, đó là nơi CPU có thể tham chiếu và truy nhập tới trong một chu kỳ bus. Vì xử lý có N bit địa chỉ có thể đánh địa chỉ được 2^N khu vực trên bộ nhớ.

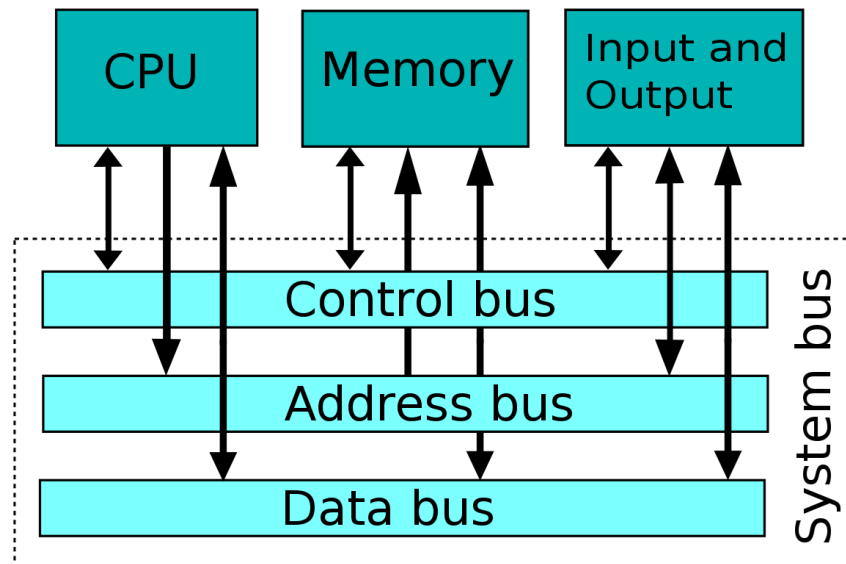


CPU Bus và I/O trên bus



□ Bus dữ liệu

- Các kênh truyền tải thông tin theo hai chiều giữa CPU và bộ nhớ hoặc các thiết bị ngoại vi vào ra.
- Được điều khiển bởi CPU để đọc/ghi dữ liệu hoặc mã lệnh.
- Độ rộng của bus dữ liệu xác định được lượng dữ liệu truyền tải và khả năng xử lý của CPU.

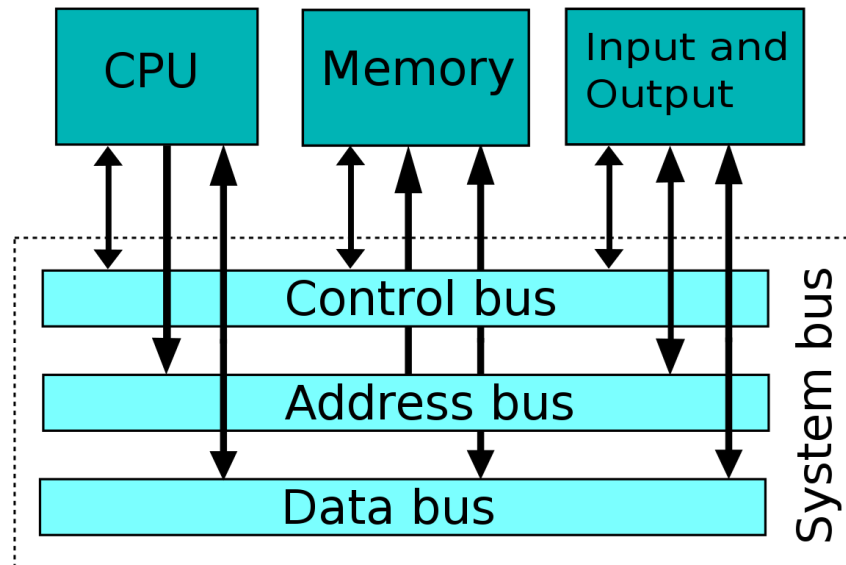


CPU Bus và I/O trên bus



□ Bus điều khiển

- Truyền tải tín hiệu điều khiển giữa các thành phần trong CPU.
- Trao đổi tín hiệu điều khiển giữa CPU và các ngoại vi: bộ nhớ, thiết bị nhập xuất...

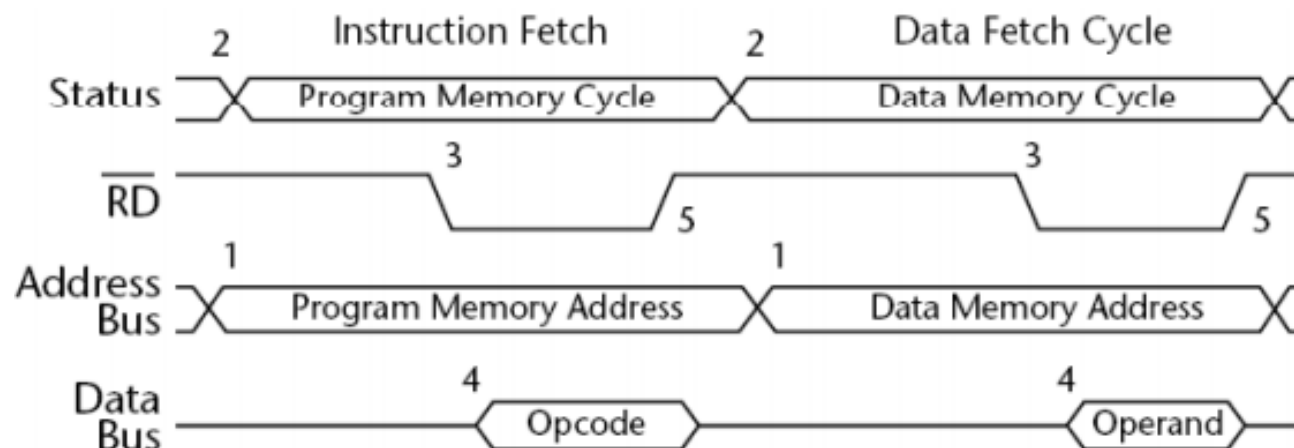


Điều khiển truy cập Bus



❖ Hoạt động truy cập đọc (read)

1. CPU xuất lên bus địa chỉ thông tin xác định khu vực bộ nhớ cần truy nhập để đọc dữ liệu ra (bộ nhớ hoặc ngoại vi)
2. CPU điều khiển tín hiệu trên bus điều khiển để kích hoạt bộ nhớ/ngoại vi xuất dữ liệu lên bus dữ liệu.
3. CPU chốt và đọc dữ liệu về bộ nhớ trong và kết thúc hoạt động truy nhập đọc

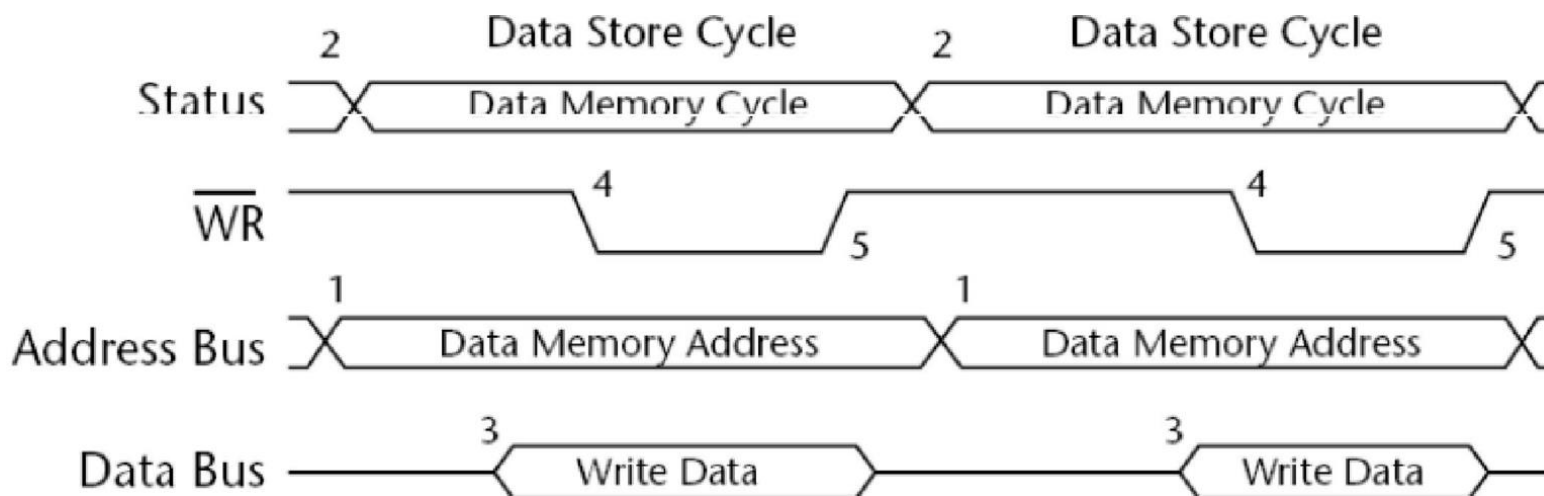


Điều khiển truy cập Bus



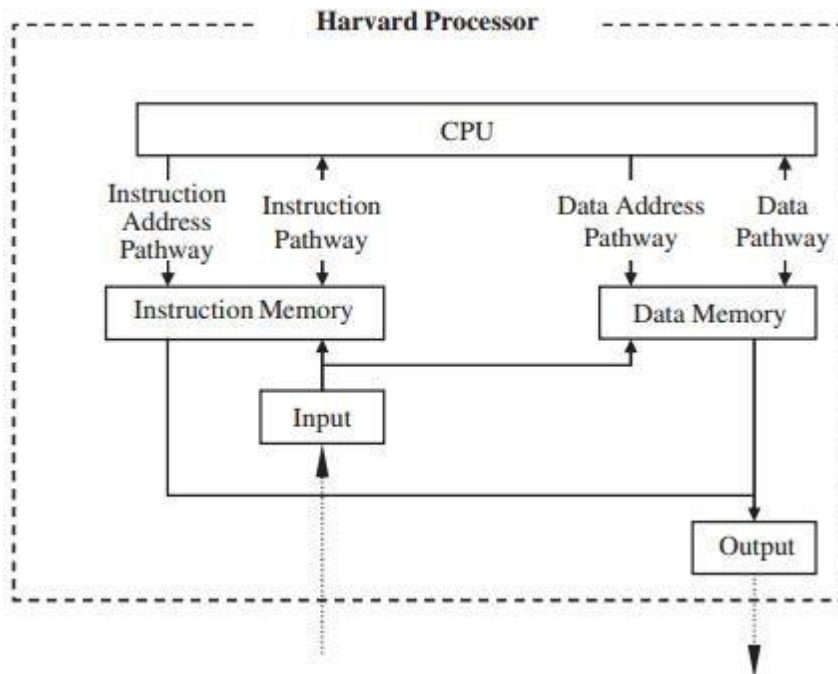
❖ Hoạt động truy cập ghi (write)

1. CPU xuất địa chỉ khu vực bộ nhớ cần truy nhập để viết dữ liệu vào lên bus địa chỉ.
2. Sau khi thông tin địa chỉ đã được thiết lập trên bus địa chỉ CPU sẽ điều khiển xuất dữ liệu cần viết vào khu vực nhớ lên bus dữ liệu.
3. CPU điều khiển tín hiệu trên bus điều khiển để kích hoạt bộ nhớ/ ngoại vi chốt và ghi dữ liệu vào khu vực nhớ đã chỉ ra và kết thúc hoạt động truy nhập viết

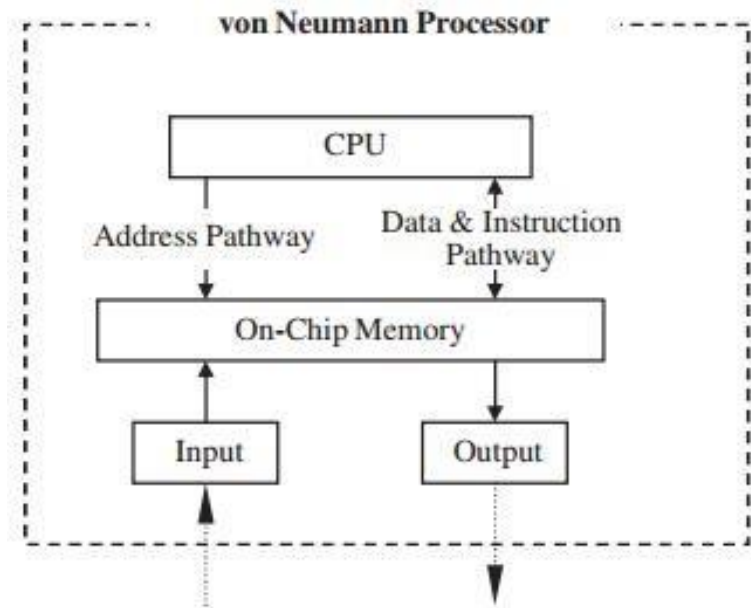




□ Tổng quan 2 loại kiến trúc bộ nhớ



Kiến trúc von Harvard



Kiến trúc von Neumann

Cấu trúc và phân loại hệ thống bộ nhớ



- ❑ Bộ nhớ chương trình – ROM
 - ❑ PROM: Bộ nhớ chỉ có thể đọc và được sử dụng để lưu mã chương trình, chỉ được viết vào duy nhất một lần
 - ❑ EPROM: Được cấu tạo bởi một mảng transistor khả trình. Bộ nhớ có thể ghi và xoá được bằng tia cực tím và có thể được lập trình lại
 - ❑ EEPROM: Giống EPROM nhưng có thể xoá trực tiếp bằng điện và thực hiện trên mạch ứng dụng không cần thiết bị nạp điện và thực hiện trên mạch ứng dụng không cần thiết bị nạp chuyên dụng
 - ❑ FLASH: Giống như EEPROM nhưng có thể xoá theo vùng hoặc khối byte

Cấu trúc và phân loại hệ thống bộ nhớ



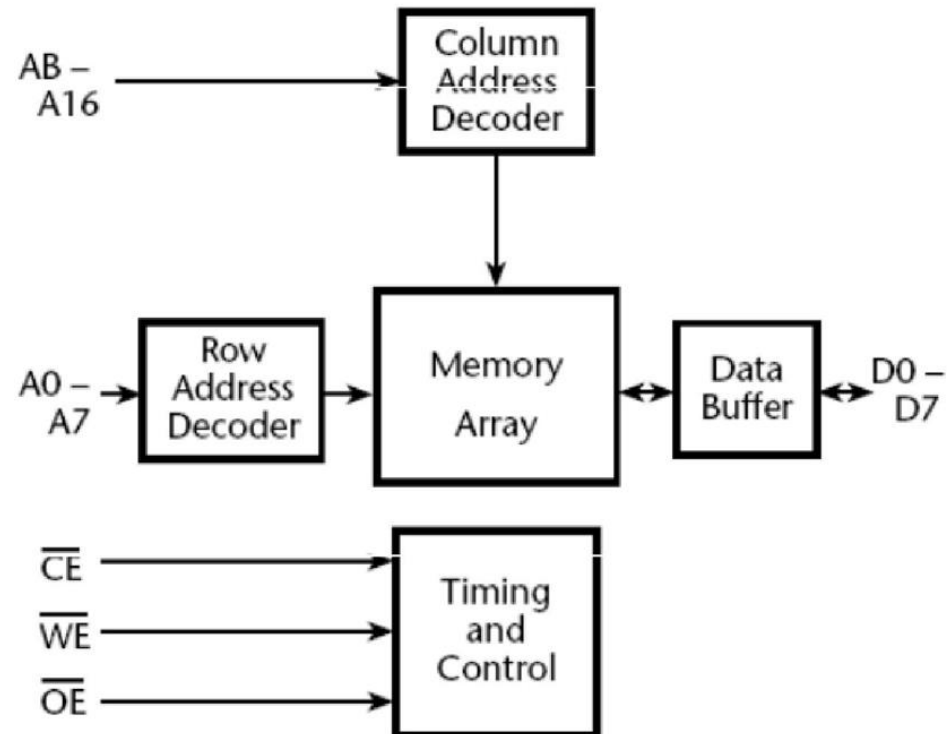
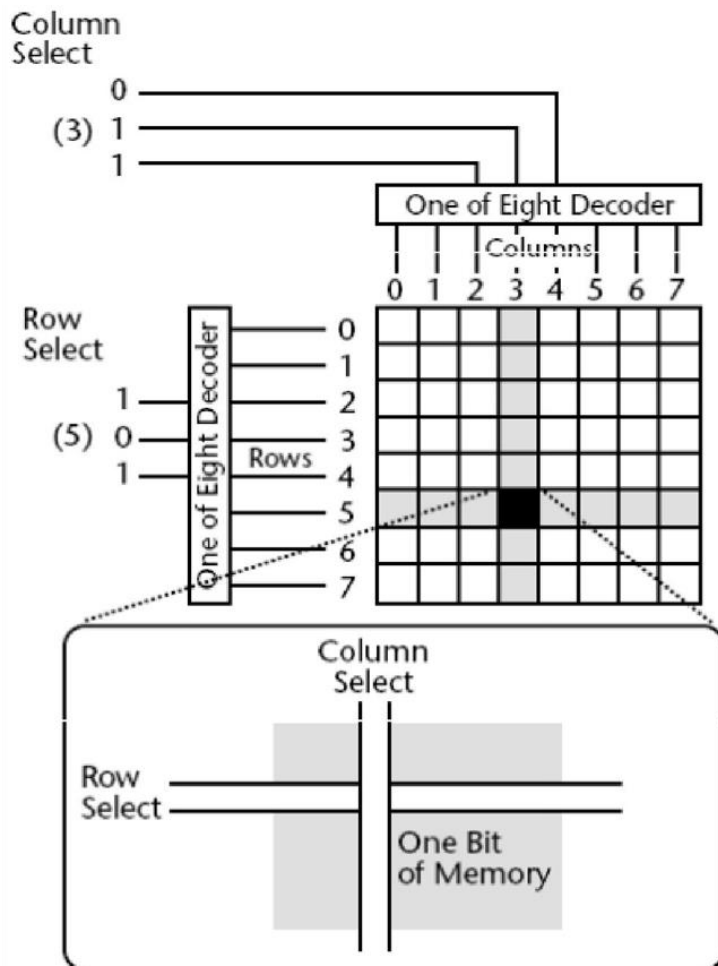
□ Bộ nhớ dữ liệu– RAM

- DRAM: Hay còn gọi là RAM động là loại bộ nhớ dữ liệu có thể truy nhập đọc/ghi. DRAM được cấu thành bởi các phần tử tụ điện nhỏ để lưu trữ trạng thái thông tin. Hoạt động lưu trữ thông tin thực hiện theo nguyên lý phóng nạp của các tụ điện. Trạng thái thông tin được lưu giữ chỉ khi tụ điện được duy trì nạp đầy vì thực hiện theo nguyên lý phóng nạp của các tụ điện. Trạng thái thông tin được lưu giữ chỉ khi tụ điện được duy trì nạp đầy vì vậy nó phải thường xuyên được refresh trong quá trình hoạt động.
- SRAM: Hay còn gọi là RAM tĩnh là loại bộ nhớ dữ liệu có thể truy nhập đọc/ghi. SRAM được cấu thành từ các phần tử Trigger Flip-Flop để lưu trữ trạng thái thông tin

Cấu trúc và phân loại hệ thống bộ nhớ



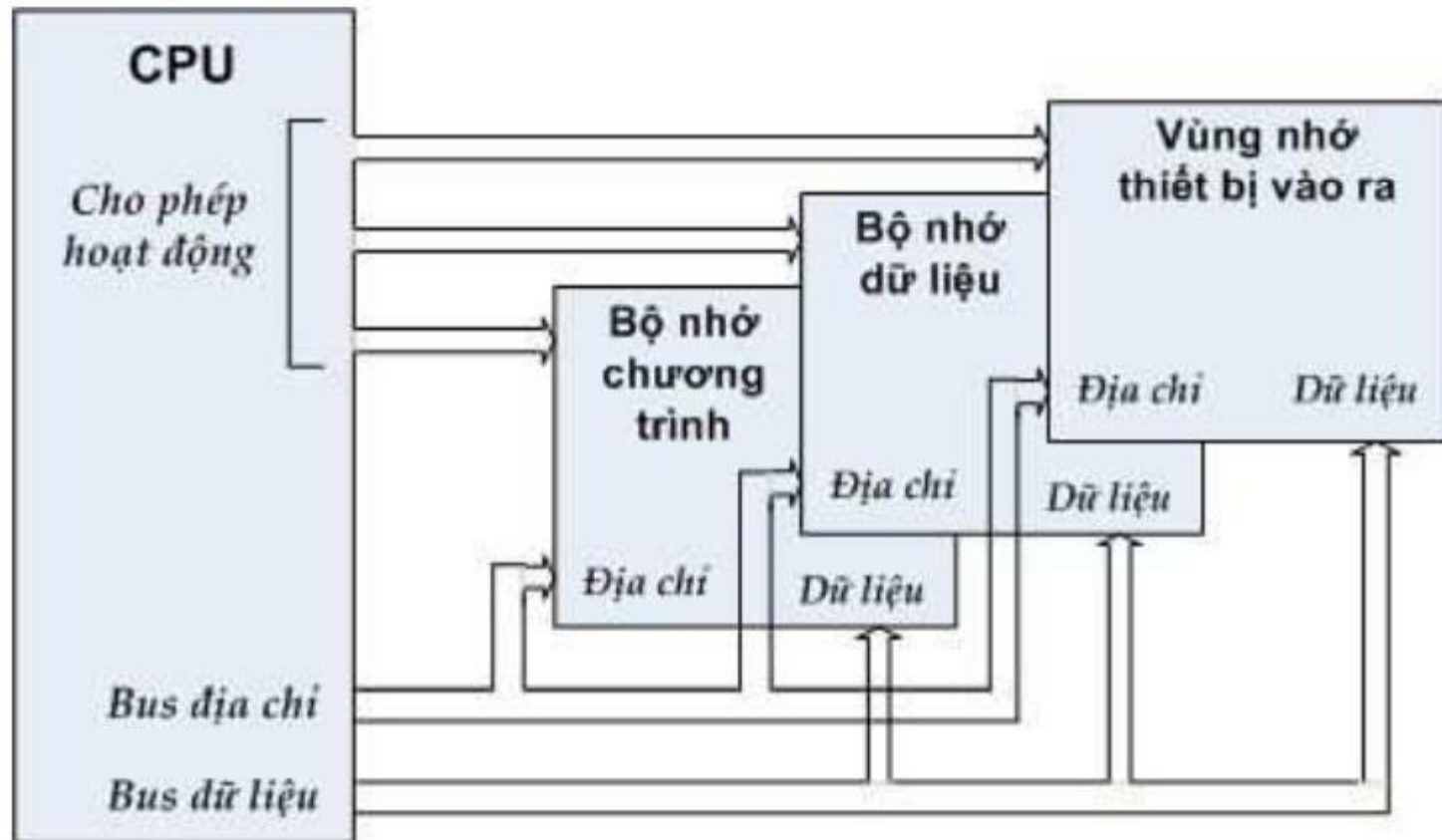
□ Tổ chức và nguyên lý hoạt động truy cập RAM





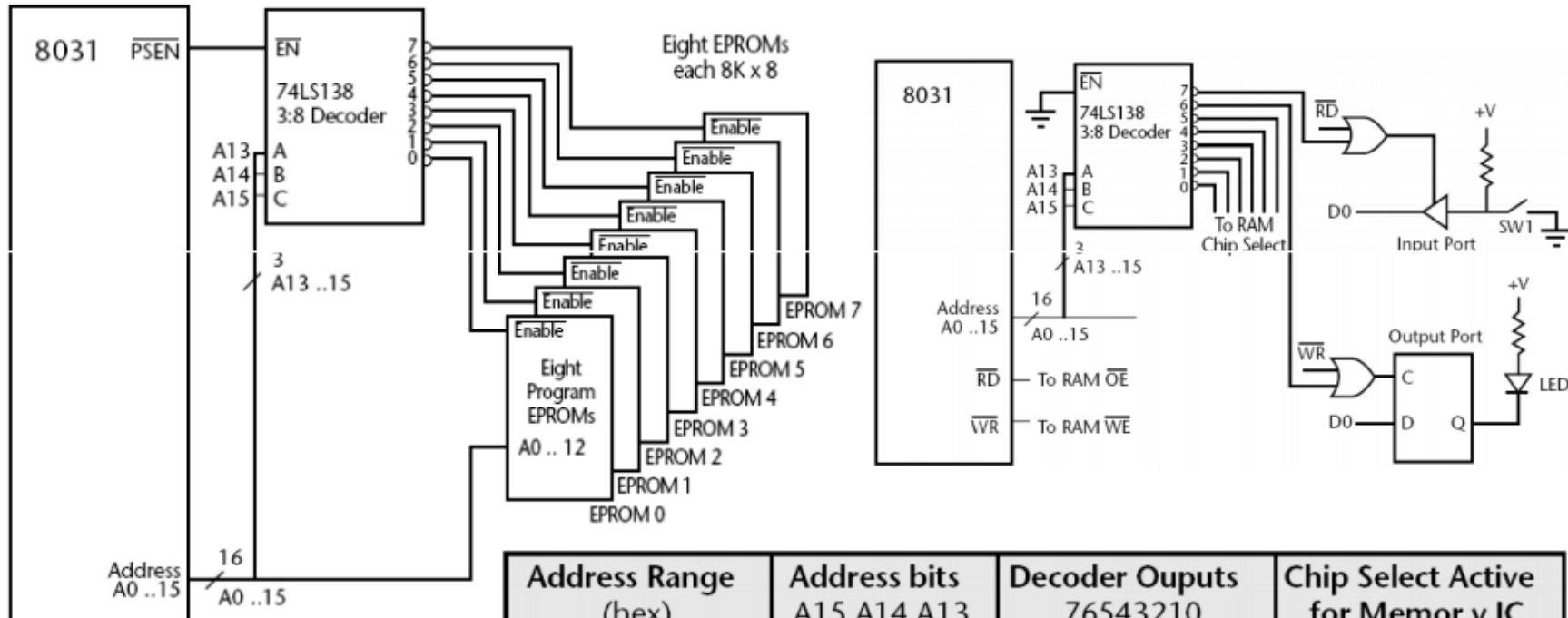
- ❑ Một số vùng bộ nhớ cơ bản có thể được phân vùng để quản lý:
 - ❑ Không gian địa chỉ vùng nhớ chương trình
 - ❑ Không gian địa chỉ vùng nhớ dữ liệu
 - ❑ Không gian địa chỉ vào ra ngoại vi
 - ❑ Không gian địa chỉ vùng nhớ ngăn xếp
- ❑ Các vùng bộ nhớ có thể được quản lý tách biệt hoàn toàn hoặc trùng nhau tùy thuộc từng kiến trúc cụ thể.

Phân vùng & tổ chức không gian địa chỉ





Phân vùng & tổ chức không gian địa chỉ



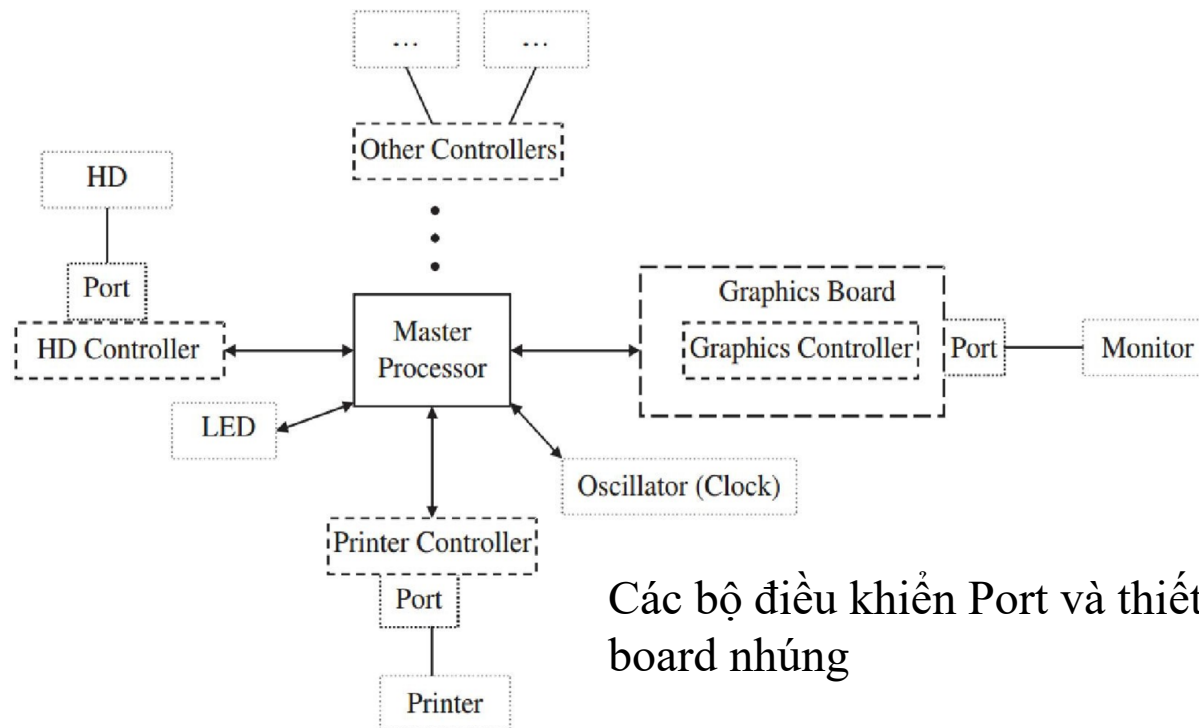
Address Range (hex)	Address bits A15 A14 A13	Decoder Outputs 76543210	Chip Select Active for Memory IC
0000 - 1FFF	0 0 0	11111110	EPROM 0
2000 - 3FFF	0 0 1	11111101	EPROM 1
4000 - 5FFF	0 1 0	11111011	EPROM 2
6000 - 7FFF	0 1 1	11110111	EPROM 3
8000 - 9FFF	1 0 0	11101111	EPROM 4
A000 - BFFF	1 0 1	11011111	EPROM 5
C000 - DFFF	1 1 0	10111111	EPROM 6
E000 - FFFF	1 1 1	01111111	EPROM 7

Các thiết bị nhập xuất I/O



□ Tổng quan:

Hệ thống nhúng về tổng quát có thể coi là một máy vi tính hay hệ vi xử lý hoàn chỉnh. Mỗi máy vi tính đều có cấu trúc chung gồm khối xử lý trung tâm CPU, bộ nhớ và các mạch vào ra I/O (Board I/O).



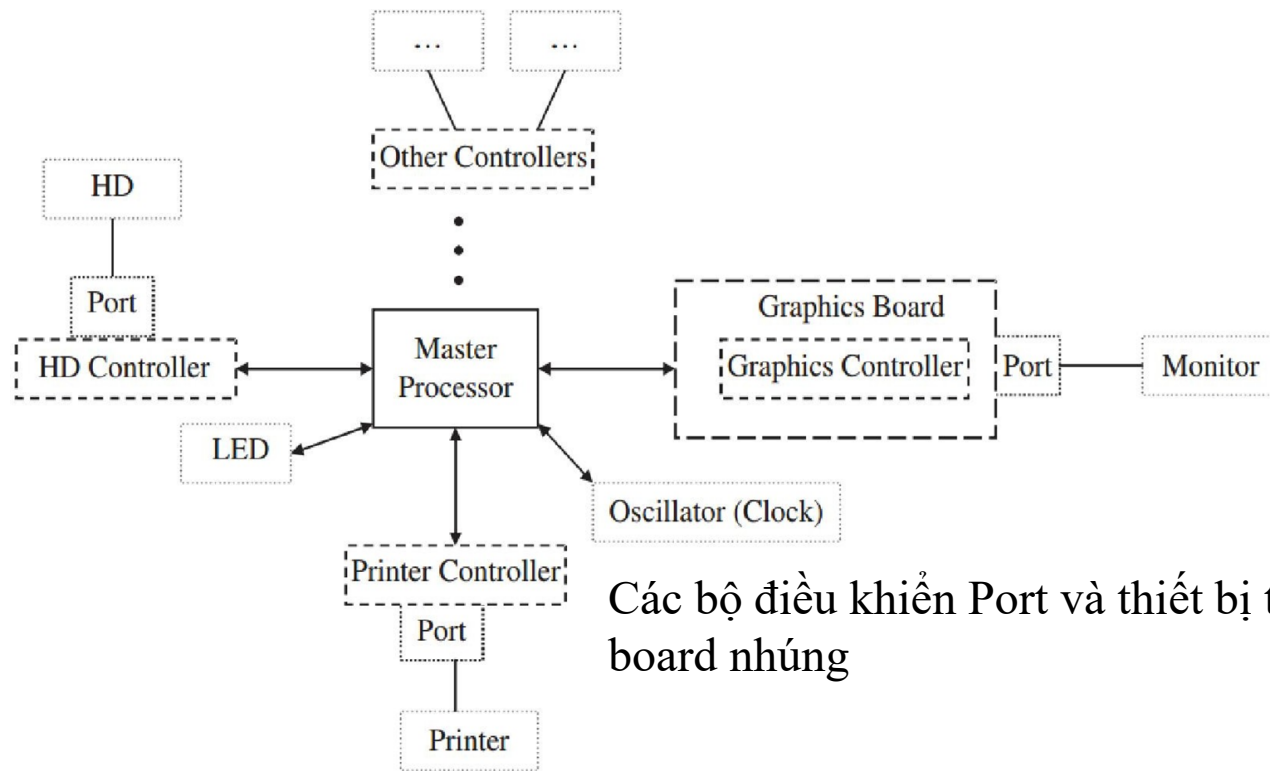
Các bộ điều khiển Port và thiết bị trên một board nhúng

Các thiết bị nhập xuất I/O



□ Tổng quan:

Hệ thống nhúng về tổng quát có thể coi là một máy vi tính hay hệ vi xử lý hoàn chỉnh. Mỗi máy vi tính đều có cấu trúc chung gồm khối xử lý trung tâm CPU, bộ nhớ và các mạch vào ra I/O (Board I/O).



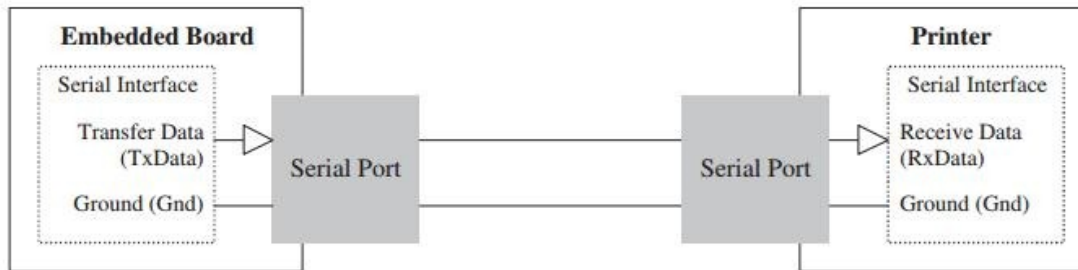
Các bộ điều khiển Port và thiết bị trên một board nhúng

Truyền thông nối tiếp – Serial I/O

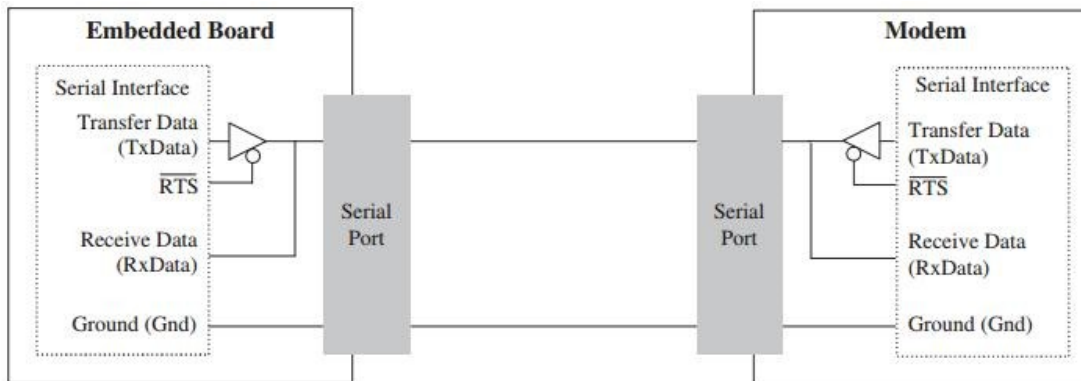


- ❑ Nhập/xuất trên board theo kiểu truyền và nhận dữ liệu nối tiếp được tạo thành từ các thành phần trong đó dữ liệu (ký tự) được lưu trữ, di chuyển, và nhận từng bit nối tiếp nhau
- ❑ Dữ liệu có thể được truyền đi giữa hai thiết bị ở một trong ba hướng: trong một chiều, theo cả hai chiều (nhưng phải phân chia thời gian truyền vì chúng chia sẻ các đường truyền tải như nhau), và trong cả hai hướng cùng một lúc.

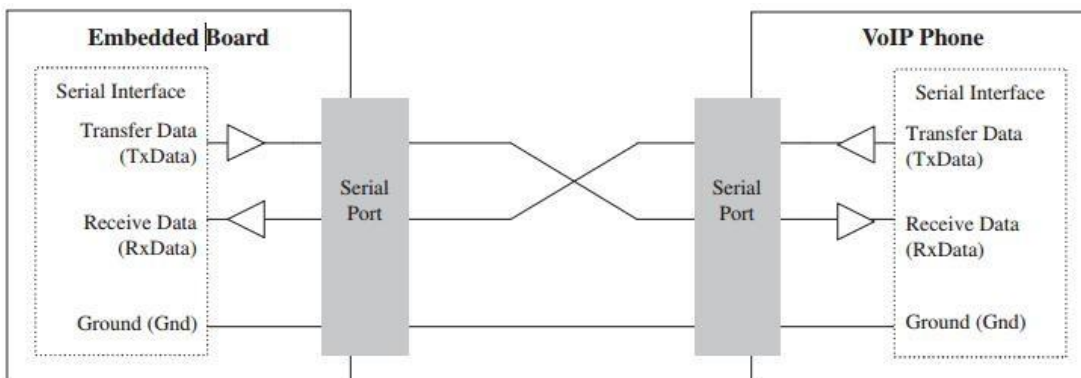
Truyền thông nối tiếp – Serial I/O



Hệ thống truyền nối tiếp 1 hướng đơn giản



Hệ thống truyền theo half-duplex



Hệ thống truyền theo full-duplex

Truyền thông nối tiếp – Serial I/O



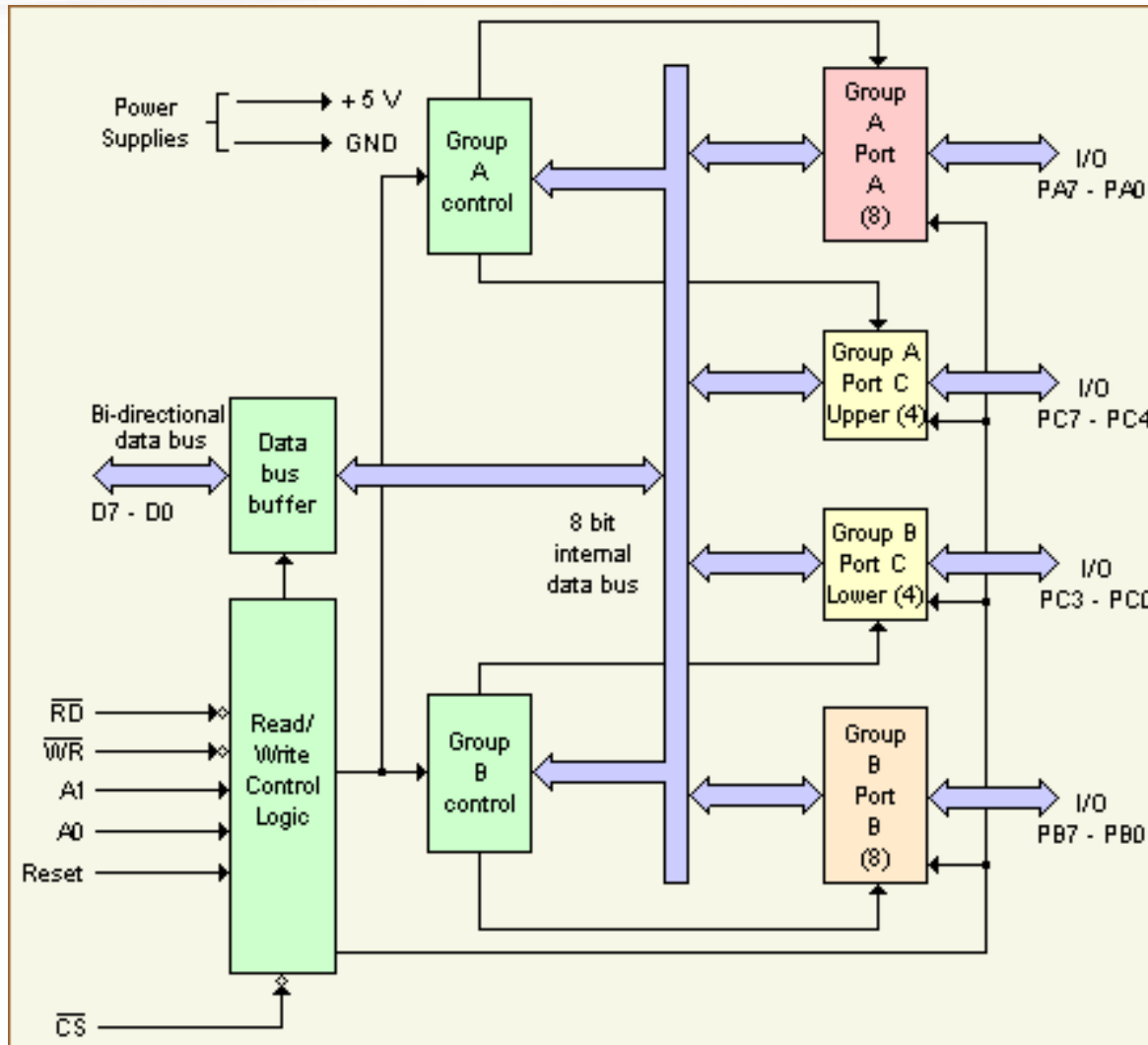
- Cổng nối tiếp được sử dụng để truyền dữ liệu hai chiều giữa máy tính và ngoại vi, có các ưu điểm sau:
 - Khoảng cách truyền xa hơn truyền song song.
 - Số dây kết nối ít.
 - Có thể truyền không dây dùng hồng ngoại.
 - Có thể ghép nối với vi điều khiển hay PLC (Programmable Logic Device).
 - Cho phép nối mạng.
 - Có thể tháo lắp thiết bị trong lúc máy tính đang làm việc.
 - Có thể cung cấp nguồn cho các mạch điện đơn giản

Truyền thông song song– Parallel I/O



- ❑ Nhập/xuất trên board theo kiểu truyền và nhận dữ liệu song song được là kiểu truyền trong đó dữ liệu (ký tự) được lưu trữ, di chuyển, và nhận theo từng cụm bit (4,8,16,32 bit,...)
- ❑ Một số IC hỗ trợ kiểu truyền song song như: 8255

Truyền thông song song– Parallel I/O

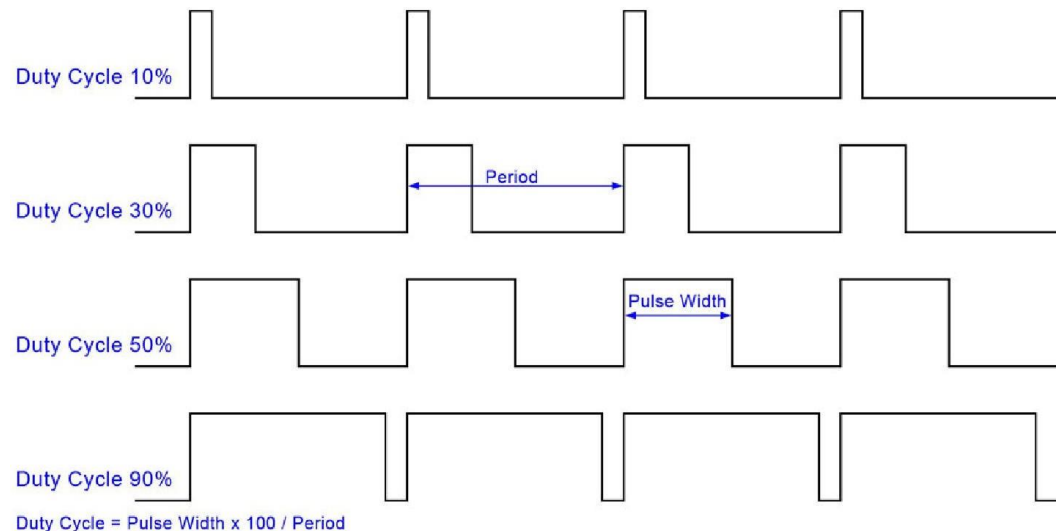
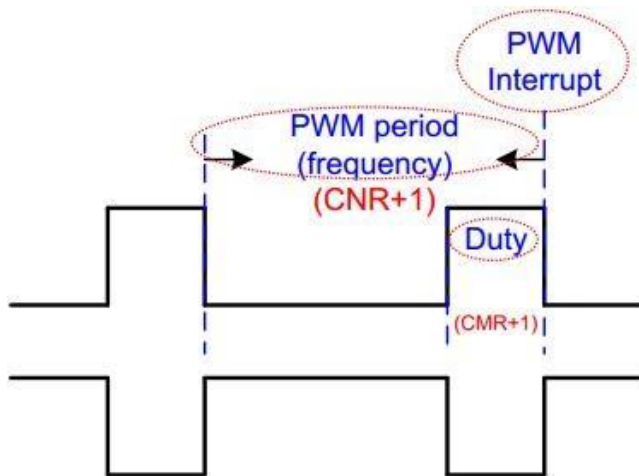


Giản đồ chức năng của 82C55A

Bộ điều chế độ rộng xung PWM



- Cho phép điều chế độ rộng xung trong các ứng dụng điều khiển điện tử công suất nhất là trong lĩnh vực điều khiển động cơ.



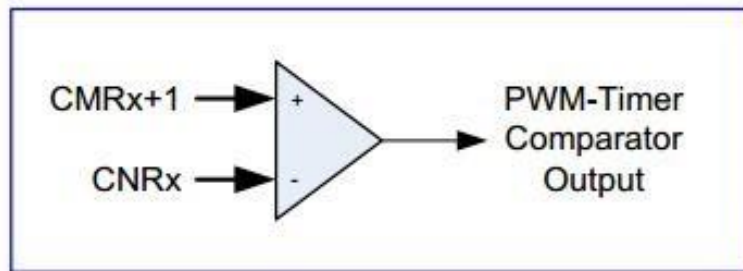
- Duty cycle là tỷ lệ phần trăm mức cao.
- Period là chu kỳ xung.
- Pulse width là giá trị mức cao so với period.

Bộ điều chế độ rộng xung PWM

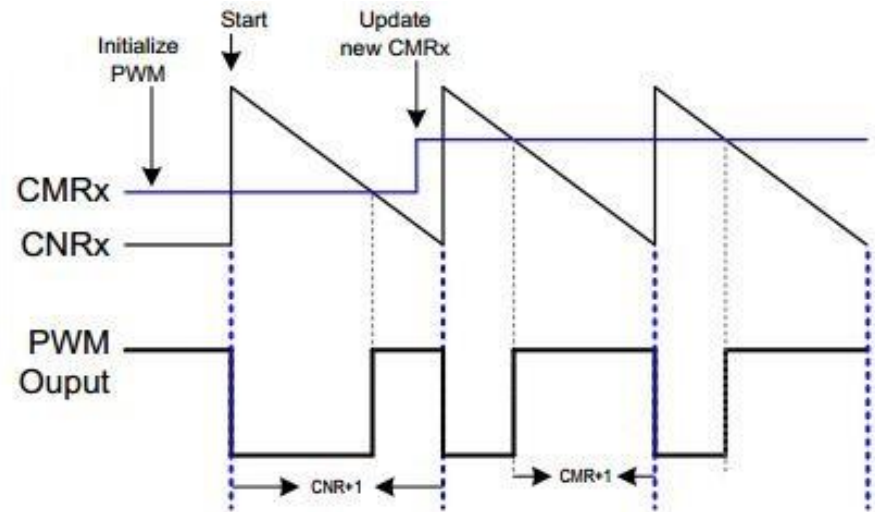


❑ Ví dụ PWM trên NUC140:

- ❑ **PWM Frequency = $\text{PWM}_{xy_CLK} / (\text{prescale} + 1) * (\text{clock divider}) / (\text{CNR} + 1)$;**
where $xy = 01, 23, 45$ or 67 , the selected PWM channel.
- ❑ **Duty ratio = $(\text{CMR} + 1) / (\text{CNR} + 1)$.**



$\text{CMRx} + 1 \geq \text{CNR}$: PWM output high.
 $\text{CMRx} + 1 < \text{CNR}$: PWM output low



GPIO

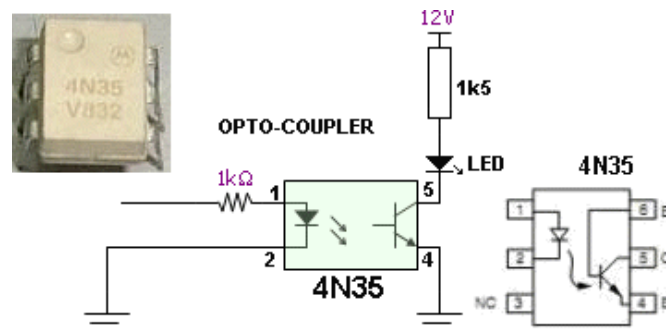


- ❑ Nhiều bộ xử lý nhúng có một số chân I/O mục đích chung (general purpose I/O - GPIO), cho phép các phần mềm để có thể đọc hoặc ghi các cấp điện áp đại diện cho logic 1 hoặc 0. Nếu bộ vi xử lý được cấp điện áp V_{DD} , ở chế độ tích cực cao mức logic 1 sẽ tương ứng một điện áp gần V_{DD} , và một điện áp gần bằng không đại diện cho mức logic 0.
- ❑ Trong nhiều mẫu thiết kế, một pin GPIO có thể được cấu hình để là một đầu ra. Điều này cho phép phần mềm sau đó ghi vào một thanh ghi của bộ nhớ ánh xạ để đặt điện áp đầu ra được hoặc là cao hay thấp. Bằng cơ chế này, phần mềm có thể kiểm soát trực tiếp các thiết bị vật lý bên ngoài.
- ❑ Nếu một pin GPIO tạo ra một điện áp đầu ra V_{DD} kết nối với 1 thiết bị có trở kháng R , thì theo định luật Ohm ta tính được dòng output của thiết bị

$$I = V_{DD}/R$$



- ❑ Một điều rất quan trọng là duy trì cách điện giữa các mạch vi xử lý và các thiết bị bên ngoài. Các thiết bị bên ngoài có thể có đặc tính nhiễu (noisy) điện sẽ làm cho các bộ xử lý không đáng tin cậy
- ❑ Thiết bị cô lập như cách ly quang (opto-isolators) có thể được sử dụng để cho phép giao tiếp giữa các vùng điện



Minh họa cách sử dụng opto

Các kỹ thuật thiết kế và debug



□ Tổng quan:

- Debug là hành động dùng một số kỹ thuật để dò và vá các lỗi xảy ra trong chương trình.
- Không có phương pháp chính tắc, tuy nhiên có một số vấn đề liên quan có thể dùng trong quá trình debug:
 - Testing
 - Phân loại bug
 - Các quá trình trong hệ thống
 - Các công nghệ được sử dụng
 - Công cụ sử dụng

Debugging và Testing



- Test và debug là một cặp đi chung với nhau
 - Test tìm lỗi, debug định vị và sửa chữa chúng
 - Quy trình test -> debug là một chu trình tuần hoàn
 - Test và debug không nên được thực hiện bởi cùng một người

Tại sao debug lại khó?



- ❑ Có thể không có mối quan hệ rõ ràng giữa biểu hiện bên ngoài và nguyên nhân bên trong
- ❑ Triệu chứng và nguyên nhân có thể ở các phần tách rời trong chương trình
- ❑ Sự thay đổi trong chương trình (tính năng mới, vá lỗi khác) có thể làm bugs bị che đậy kỹ hơn
- ❑ Một số lỗi nguyên nhân do sự hiểu lầm của con người hơn là lỗi thiết kế
- ❑ Một số lỗi xảy ra ngẫu nhiên, khó lập lại để khoanh vùng

Phương pháp debug



- ❑ Phòng cháy không bằng chữa cháy!
- ❑ Thiết kế chương trình có dự trù cho debug
 - Khi viết code, bạn phải nghĩ tới bạn sẽ debug nó trong tương lai như thế nào
 - Viết test cases ngay khi viết code
 - Nếu một biến/event nào đó nên có giá trị true thì **phải** kiểm tra nó
 - Tạo những hàm giúp mô hình hóa dữ liệu của bạn
 - Bắt đầu test chương trình từ sớm và từng hàm.

Ép Lỗi – Fault Injection



- ❑ Nhiều bug chỉ xảy ra trong các trường hợp bất thường
- ❑ Tạo ra những trường hợp trong đó các tiến trình bị thất bại để bắt những lỗi đó
 - Ví dụ: mở file, ghi file, cấp phát bộ nhớ...
- ❑ Thiết kế một bộ test những dữ liệu bất thường

Tìm và sửa lỗi



- ❑ Để tăng chất lượng phần mềm, cơ chế người dùng có thể báo bug nên được cung cấp
- ❑ Các lỗi thường có xu hướng có thể lặp lại nên một khi nó xuất hiện người lập trình nên quan tâm đến nó ngay

Các loại bugs



- ❑ Một số loại lỗi và biểu hiện
 - **Compile time:** lỗi cú pháp, chính tả, loại biến.
 - Thường xuất hiện khi compile
 - **Design:** thuật toán sai.
 - Sai output
 - **Program logic** (if/else, thoát loop, case, etc).
 - Sai output
 - **Memory:** con trỏ null, giới hạn của array, sai loại.
 - Runtime exceptions
 - **Lỗi giao tiếp giữa các modules, threads, programs** (đặc biệt là khi giao tiếp với các tài nguyên dùng chung: sockets, files, memory, etc).
 - Runtime Exceptions
 - **Những điều kiện không được định nghĩa:** lỗi của các thành phần phần mềm nằm ở lớp dưới (ví dụ lớp mạng)
 - Incomplete functionality
 - **Deadlocks:** nhiều processes cùng tranh dành tài nguyên

Tiến trình debug



- ❑ Quá trình debug thông thường của một kỹ sư
 - Xác định trường hợp tương đối chắc chắn làm xảy ra lỗi
 - Loại trừ và cách ly vấn đề thành những phần nhỏ để dễ xác định
 - So sánh và xác định các giá trị cần có của từng thành phần -> Xác định điểm cần thay đổi
 - Thay đổi chương trình, và xem hiện tượng xảy ra
 - Theo dõi các sự thay đổi của thành phần này có làm ảnh hưởng tới các module khác không
 - Nếu đúng xác nhận fix bug thành công và cập nhật tài liệu

Công cụ dùng trong debug



❑ Debugger

- Một công cụ phần mềm/phần cứng được sử dụng để phát hiện các mã nguồn của chương trình bị lỗi, bằng cách hỗ trợ thực hiện việc chạy chương trình theo từng bước(step by step) và xem nội dung của các biến mã

❑ Debugger không phải là IDE

- Mặc dù chúng có thể được tích hợp chung, nhưng chúng là 2 phần khác nhau
- Một số debugger cần phần cứng hỗ trợ.



UIT-HCM



Thank You !